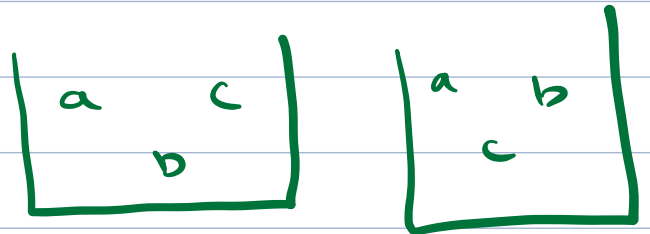


- Introduction to Strings
- Substrings
- Toggle Each Char
- Longest Palindromic Substring
- Immutability in Strings

Strings

~~Collection~~ of characters



sequence of characters

array of characters

"Hello world!"

"Apurva"

Note - Strings are represented using double quotes. In some cases, single quotes also; but mostly single quotes for characters

Char $\xrightarrow{\text{Number}}$ ASCII $\xrightarrow{\text{Computer}}$ Binary

Characters → Building block of string

↳ single symbol that represents a letter, digit, special symbol

'a', 'A', 'B', '1', '-', '@'

"11" → string (2 chars)

char is mapped to an int (ASCII value) ⁰⁻²⁵⁵

A → 65

B → 66

C → 67

⋮

Z → 90

a → 97

b → 98

c → 99

⋮

z → 122

0 → 48

1 → 49

2 → 50

⋮

9 → 57

10 → 58

"10" → 49 48

① char ch = (char) 65

print (ch) // A

int → char

48 → 10

② char ch = 65

print (ch) // Error

Big → small (Error)
(Loss of info.)

③ char ch = (char) ('a' + 1)

print (ch) // b

char + int
'a' + 1
97 + 1 = 98

④ int x = 'z'

print (x) // 122

Char → int
Small → Big ✓

$ch = (char)(ch + 1)$

```
for (char ch = 'a' ; ch ≤ 'z' ; ch++) <
    |
    > print (ch)
```

```
for (int ch = 97 ; ch ≤ 122 ; ch++) <
    |
    > print ((char)ch)
```

string

1. Given a ~~char~~ string `ch[]`, toggle every character.
Note: input has small & capital chars.

`ch[] : "Hello"`
ans → **HELLO**

`I/P : "aDgbHJe"`
ans → **AdGbHjE**

'A' → 65		'a' → 97	65 97
'B' → 66		'b' → 98	'A' ⁺³² → 'a'
'C' → 67		'c' → 99	66 98
⋮		⋮	'B' ⁺³² → 'b'
⋮		⋮	
'Z' → 90		'z' → 122	

upper case $\xrightarrow{+32}$ Lower case

upper case $\xleftarrow{-32}$ Lower case

Approach : Go to every char in string, check if it is uppercase or lowercase, add or subtract 32 accordingly.

```
// string s
int n = s.size()
for (i = 0 ; i < n ; i++) {
    char ch = s[i]
    if (ch >= 'A' && ch <= 'Z') {
        s[i] = (char)(ch + 32)
    }
    else {
        s[i] = (char)(ch - 32)
    }
}
return s
```

TC: $O(N)$
SC: $O(1)$

$ch \geq 'A' \quad \&\& \quad ch \leq 'Z'$
 $ASCII \geq 65 \quad \&\& \quad ASCII \leq 90$

★ In Java and Python, a string is immutable

```

// string s
int n = s.size()
for (i = 0 ; i < n ; i++) {
    char ch = s[i]
    if (ch >= 'A' && ch <= 'Z') {
        ans = ans + (char)(ch + 32)
    }
    else {
        ans = ans + (char)(ch - 32)
    }
}
return ans

```

String ans = ""

String ans = "Hello"

ans += 'a'

ans = ans + 'a'

ans
13K
17K

X

ans = "Hello"

#13K

ans = "Helloa"

#17K

1 addition $\rightarrow O(N)$

N additions $\rightarrow N \times N = N^2$

highly unoptimal in java
→ gives TLE

How to handle in Java?

```
// string s
char arr[] = s.toCharArray()
for (i=0 ; i < n ; i++) {
    char ch = arr[i]
    if (ch >='A' && ch <='Z') {
        arr[i] = (char)(ch + 32)
    }
    else {
        arr[i] = (char)(ch - 32)
    }
}
return new String(arr)
```

S = "Hello"

char arr = H | e | l | l | o

$N + N + N = 3N$

TC: $O(N)$

SC: $O(N)$

↓
char arr[]

abcd4

String ans = ""

ans += 'a'

ans += 'b'

ans += 'c'

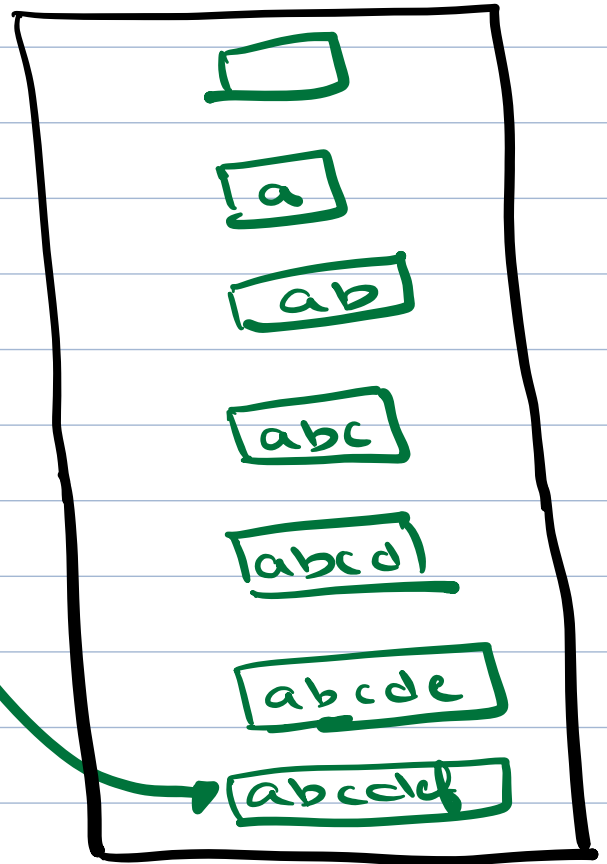
ans += 'd'

ans += 'e'

ans += 'f'

ans

String pool



Apart from initial string object, 6 new String objects were created.

Substring: contiguous sequence of characters in a string

- ① Continuous part of string
- ② Full string → substring
- ③ single character → substring

substring
↓
Subarray

⁰ ¹ ²
"a b c"

ans = b

a	b	c
ab	bc	
abc		

$$\text{Cnt of substrings} = \frac{N(N+1)}{2}$$

"b⁰a¹c²d³"

N=4

$$\text{cnt} = \frac{4 \times 5}{2} = 10$$

b a c d
 b a a c c d
 b a c a c d
 b a c d

Palindrome: string that reads the same forward and backward

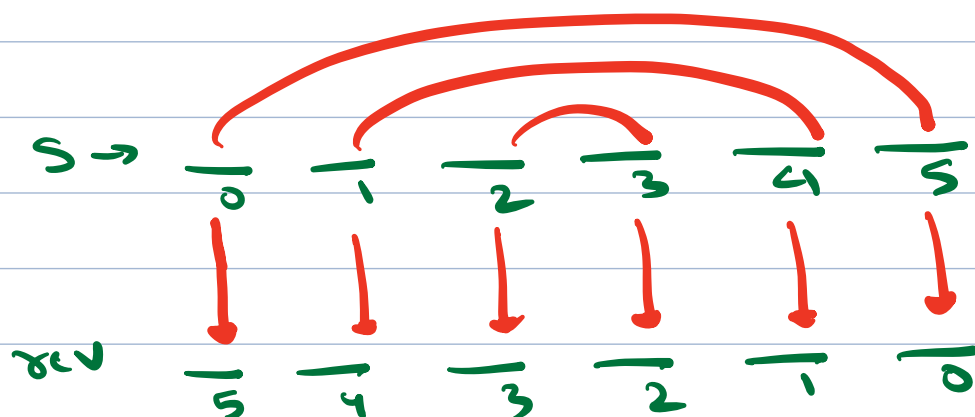
2. Check whether the given string is palindrome or not.

S = "madam"

ans: true

S = "hello"

ans: false



S = c
STOP

s e
 m a d a m
 0 1 2 3 4

s r e
 STOP c a d d a c
 0 1 2 3 4 5

bool checkPalindrome (string A) <

int n = A.size()

int s = 0, e = n-1

while (s <= e) <

if (A[s] == A[e]) <

s++ e--

else <

return false

return true

Tc: O(N)
Sc: O(1)

2 char → 1 itr
N char → $\frac{N}{2}$ itr

2. Check whether the given substring is palindrome or not.

I/P : string, i, j

S = "anamaclamspe"

i = 3

j = 7

ans : true

10:40

bool checkPalindrome (string A, int i, int j) {

int s = i, e = j

while (s <= e) {

if (A[s] == A[e]) {

s++ e--

} else {

return false

} return true

i j
↓ ↓
0 n-1

TC: $O(N^2)$

SC: $O(1)$

Q3. Given a string s, calculate length of longest palindromic substring.

s = a b a c a b

ans → 5

Ans
↓ ↓
1 2
worst Best

s = f e a c a b a c a b g f

ans → 7

S = ^{0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16}
a d a e b c d f d c b e t g g t e

ans → 9

Approach 1: Go to all substrings → check palindrome. If yes, compare its len with ans

// String A

[s e] = e - s + 1

int ans = 0

int n = A.size()

for (s = 0 ; s < n ; s++) {

for (e = s ; e < n ; e++) {

// substring (s e)

if (checkPalindrome(A, s, e) == true) {

ans = max(ans, e - s + 1)

return ans

1 substring → $O(N)$
 N^2 substrings → $N^2 \times N$
 $= O(N^3)$

TC : $O(N^3)$

SC : $O(1)$

Optimized Approach

① Odd length palindromic substring

Treat every char as centre of palindrome, expand in both directions as much as possible

s e
m a d a m
d
a d a
m a d a m

// string A

int ans = 0

int n = A.size()

for (i=0 ; i < n ; i++) <

// Assume ith char is centre

int s = i , e = i

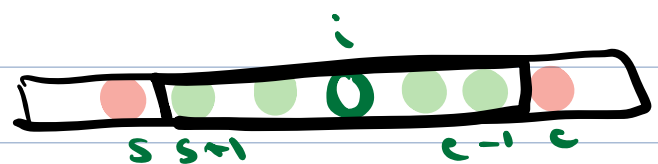
while (s >= 0 && e < n && A[s] == A[e]) <

s-- e++

len = e - s - 1

ans = max(ans, len)

[a b] = b - a + 1



1 centre $\rightarrow O(N)$
N centre $\rightarrow O(N^2)$

[s+1 $\rightarrow$ e-1]
= e-1 - (s+1) + 1
= e-1-s-x+1

② Even length palindromic substring

Treat every 2 consecutive chars
as centre of
palindrome, expand
in both directions as
much as possible

^s m a dd a m ^e

dd
adda
maddam

```
for (i=0 ; i < n ; i++) {
```

```
    // Assume ith and (i+1)th char is centre
```

```
    int s = i , e = i+1
```

```
    while (s >= 0 && e < n && A[s] == A[e]) {
```

```
        s-- e++
```

```
        len = e - s - 1
```

```
        ans = max(ans, len)
```

```
}
```

```
return ans
```

TC: $O(N^2)$

SC: $O(1)$

String Immutability

Java, C#, JS, Python, Go



strings are immutable → its value can't be changed

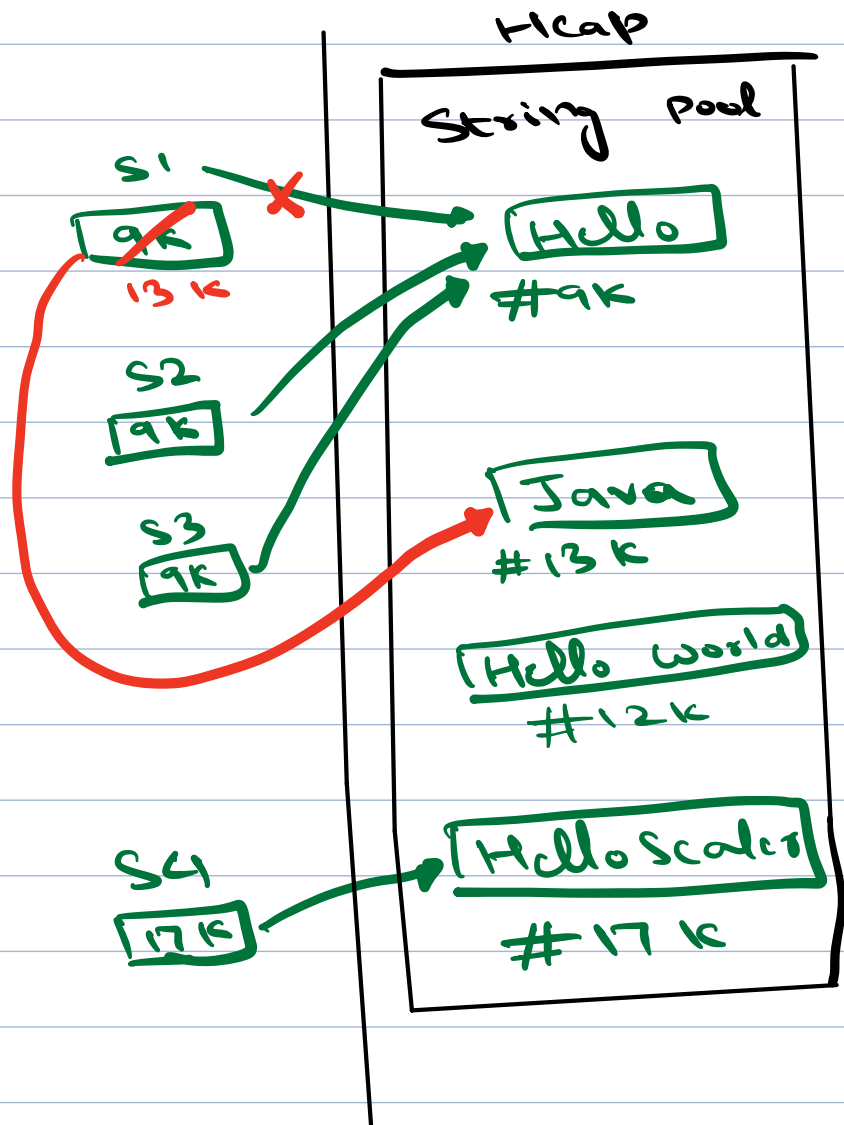
String s1 = "Hello"

String s2 = "Hello"

String s3 = s1

s1 = "Java"

s2.concat("world")



String s4 = s3.concat("Scaler")

s1 → Java

s2, s3 → Hello

s4 → HelloScaler

StringBuilder / String Buffer

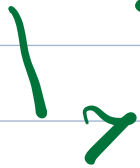


maintain a mutable sequence of characters

Add a character → TC is $O(1)$ on avg

```
String Builder sb = new StringBuilder()
```

```
for (int i = 0; i < n; i++) {  
    sb.append('a')
```



```
String res = sb.toString()
```

